MODULE 22 · DOMAIN 3 - DEPLOYMENT AND ORCHESTRATION

Compute and optimisation

Instance families, Inferentia, Trainium, and four ways to make a model cheaper to run.

MODULE 22 of 43 DOMAIN 3 - Deploy WEIGHT 24%

admin@sthithapragna.com

Training is a bounded cost. **Inference runs forever**, so a percentage saved compounds.

Instance families for inference

FAMILY	WHAT IT IS FOR
C	Compute optimised. Classical models, high CPU, low memory need
M	General purpose. A balanced default when nothing is extreme
R	Memory optimised. Large models or large in-memory feature sets
G	GPU, inference-oriented. Smaller deep learning models, vision
P	GPU, training-class. Large models needing serious accelerator memory
Inf	AWS Inferentia. Purpose-built inference accelerators
Trn	AWS Trainium. Built for training, and used for inference too

ON THE EXAM

Most tabular models belong on C or M. Reaching for a GPU because the model is 'important' is a common and expensive mistake.

Does this model need a GPU at all?

- **Tabular models do not.** XGBoost inference on CPU is milliseconds.
- **Deep learning models usually do**, especially vision and language.
- A GPU is only worth it if the model can **keep it busy**. An idle GPU is expensive idle silicon.
- Small batches on a GPU often perform no better than CPU, because there is nothing to parallelise.
- Remember from module 20: **serverless inference has no GPU option at all**.

> ON THE EXAM
>
> Check GPU utilisation after deploying. Consistently under 30 percent means you are paying for capacity you cannot use.

AWS Graviton

- AWS-designed **Arm-based processors**, available across the C, M and R families.
- Better **price performance** than comparable x86 instances for many workloads.
- Well suited to classical model inference, which is where most CPU inference lives.
- The container must be built for **Arm64**. An x86 image will not run.
- AWS prebuilt containers cover the common frameworks on Graviton.

> ON THE EXAM
>
> The architecture requirement is the catch. If a scenario mentions a custom x86 container, Graviton needs a rebuild first.

AWS Inferentia and Trainium

- **Inferentia** is purpose-built for inference. **Trainium** for training, and usable for inference.
- Both aim at **lower cost per unit of throughput** than comparable GPU instances.
- Each chip contains several **NeuronCores**, the hardware blocks that do the work.
- Programmed through the **AWS Neuron SDK**, which works on EC2, ECS, EKS and SageMaker AI.
- **Models must be compiled for Neuron.** You cannot drop an arbitrary model onto Inferentia unchanged.

> ON THE EXAM
>
> The compilation step is the real adoption cost. Where a model compiles cleanly, the price performance is compelling.

Inferentia or a GPU instance?

INFERENTIA, WHEN

Cost per token matters

- High, sustained inference volume.
- The model compiles cleanly for Neuron.
- A common architecture with Neuron support.
- Price performance is the priority.

GPU, WHEN

Flexibility matters

- An unusual or custom architecture.
- You need the widest framework support.
- Volume is low, so compilation effort will not repay.
- You are still iterating on the model.

THE TIEBREAKER

Inferentia pays back through volume. At low request rates the compilation effort costs more than the instance saving returns.

Changing the hardware is one lever.
Changing the **model** is the other.

Four optimisation techniques

TECHNIQUE	WHAT IT DOES
Compilation	Optimises for the target hardware, ahead of time. No accuracy loss
Quantization	Reduces numeric precision. Smaller and faster, with some accuracy cost
Speculative decoding	A small draft model proposes tokens; the large model verifies them
Fast model loading	Reduces the time taken to load a model onto an instance

ON THE EXAM

Compilation is the free one: AWS states it improves performance without a loss in accuracy. The others all trade something.

What an optimisation job involves

- You choose the model, the technique or techniques, and the target hardware.
- **For compilation and quantization, SageMaker AI chooses the instance type** for the job itself.
- **You are charged for the optimisation instance used.** An optimisation job is a real cost.
- The output is an optimised model you can deploy, or evaluate first.
- For speculative decoding you additionally choose a draft model, or take the recommended one.

> ON THE EXAM
>
> Optimisation is a job like training: it runs, it costs, and it produces an artefact. It is not a deployment setting.

Evaluating an optimised model

- After optimising, you can run an **evaluation** that reports latency, throughput and price.
- Those three together are the decision. A faster model that answers worse is not an improvement.
- **Quality is not in that evaluation.** Check it separately, using modules 17 and 18.
- For a quantized generative model, run your Bedrock-style evaluation set before and after.
- Compilation needs no quality check, because AWS documents it as accuracy-neutral.

> ON THE EXAM
>
> The built-in evaluation answers 'is it faster and cheaper'. Only your own evaluation set answers 'is it still good'.

Compilation

- Optimises the model for the chosen hardware: GPU, **Trainium** or **Inferentia**.
- **Without a loss in accuracy**, which is what separates it from quantization.
- **Ahead-of-time**, so weights do not need just-in-time compilation at deployment.
- That reduces deployment time **and auto-scaling latency**, because a new instance starts faster.
- **TensorRT-LLM** compilation is supported for NVIDIA hardware.

> ON THE EXAM
>
> The auto-scaling benefit is the one people miss. Faster model load means a scale-out event adds capacity sooner.

Quantization

- Represents weights and activations in **fewer bits**, so the model is smaller and faster.
- Named formats: **INT4-AWQ**, **INT8-SmoothQuant**, and **FP8**.
- **SmoothQuant** is mixed precision, scaling activations and weights jointly to balance their ranges.
- **FP8** against FP32: less memory, faster compute, lower power, and applicable to KV cache and attention.
- It trades **some accuracy**. Measure retrieval or task quality afterwards, do not assume.

> ON THE EXAM
>
> Quantization is how a model that will not fit in GPU memory becomes one that will. It is a capacity fix as much as a speed one.

Speculative decoding

- A small fast **draft model** generates several candidate tokens at each step.
- The large **target model verifies them**, rejecting and regenerating any it does not accept.
- It optimises **latency without compromising quality**, because the target model still decides every token.
- It breaks the sequential bottleneck of autoregressive generation, raising tokens per forward pass.
- The two knobs you control: the draft model, and **num_speculative_tokens**.

> ON THE EXAM
>
> The output is what the target model would have produced anyway. That is why it is quality-neutral, unlike quantization.

When speculative decoding works, and when it does not

- **Draft and target must share the same tokenizer and vocabulary.** Speculation works on token IDs.
- AWS recommends models from the **same architectural family**, because their predictions agree more often.
- Different architectures sharing a tokenizer will run, and **low agreement removes most of the gain**.
- Best on **predictable output**: code generation, structured extraction, templated reports.
- **Not a universal optimisation.** Effectiveness depends on prompt structure and draft quality.

> ON THE EXAM
>
> Acceptance rate is the whole mechanism. A draft model the target keeps rejecting costs you two models and buys you nothing.

Fast model loading

- Reduces the time taken to load a model onto an instance.
- That matters most at **scale-out**, when a new instance must become useful quickly.
- Large language models are tens or hundreds of gigabytes, so loading is genuinely slow.
- It also shortens the cold path on a multi-model endpoint, from module 21.
- Pairs naturally with compilation, which removes just-in-time work at load.

ON THE EXAM

Load time is invisible until traffic spikes. Then it becomes the difference between scaling in time and scaling too late.

Recommendations or manual?

RECOMMENDATIONS

AWS decides

- It analyses the model and the workload.
- Evaluates instance types and optimisations.
- Returns deployment-ready configurations.
- With real performance metrics attached.

MANUAL

You decide

- You choose which techniques to apply.
- You run the optimisation jobs.
- You evaluate the results yourself.
- More control, more work.

THE TIEBREAKER

AWS is explicit: inference recommendations is 'the recommended approach for most customers'. That phrasing is a strong exam signal.

Pre-optimized model versions

- For many models, SageMaker AI provides **several pre-optimized versions**.
- Each caters to different needs for **latency and throughput**, so you pick a profile rather than tuning.
- No optimisation job to run, and no optimisation instance to pay for.
- The fastest route from a JumpStart model to an efficient endpoint.
- Only available for models AWS has prepared. Check before planning around it.

ON THE EXAM

Before running an optimisation job, check whether a pre-optimized version already exists. It is free and immediate.

Inference Recommender

- Runs **load tests across candidate instance types** and reports latency, throughput and cost.
- **Instance recommendations** give a quick shortlist without a full load test.
- **Endpoint recommendations** run a custom load test against your own traffic pattern.
- It removes guesswork from a decision teams usually make by habit.
- Named in the exam guide, and the answer when a scenario says 'we do not know which instance type'.

> ON THE EXAM
>
> Most teams pick an instance type by copying the last project. Inference Recommender is how you replace that with measurement.

Reading the utilisation metrics

WHAT YOU SEE	WHAT IT MEANS
GPU utilisation consistently under 30 percent	Over-provisioned, or the model is data-bound. Downsize
CPU at 100 percent, GPU idle	Preprocessing is the bottleneck, not the model
Memory near the limit	Risk of OOM under load. Larger instance, or quantize
Latency rises with concurrency, utilisation low	Not compute-bound. Look at I/O or serialisation
Throughput plateaus below expectation	Instance type mismatch, or single-threaded inference code

ON THE EXAM

Row two is common and misdiagnosed constantly: people buy a bigger GPU when the preprocessing container is the constraint.

Batching, and why it matters on GPU

- A GPU processes a batch almost as quickly as a single request. Batching raises throughput sharply.
- It also raises **latency per request**, because a request waits for the batch to fill.
- **Dynamic batching** collects requests for a short window, balancing the two.
- For generation, **continuous batching** lets new requests join without waiting for the batch to finish.
- On CPU the gain is far smaller, because there is less parallelism to exploit.

> ON THE EXAM
>
> Batching is the reason a GPU endpoint's throughput can rise tenfold with no hardware change. It is also why its p99 rises.

Model server settings that affect performance

SETTING	EFFECT
Worker count	How many inference processes run per instance. Too many contend for CPU
Batch size and timeout	The dynamic batching window. Trades throughput against p99
Max concurrent requests	Backpressure. Beyond it, requests queue or are rejected
Threads per worker	Matters for CPU inference; often the cause of poor utilisation
KV cache settings	For generation: memory reserved for attention state per request

ON THE EXAM

Default model server settings are rarely optimal. This is where a well-chosen instance still underperforms.

Where each lever saves

LEVER	WHAT IT REDUCES
Right-sizing the instance	The hourly rate. Often the largest single saving
Graviton	Price per unit of CPU work, where the container supports Arm64
Inferentia or Trainium	Cost per unit of throughput, at sustained volume
Quantization	Memory footprint, so a smaller instance suffices
Speculative decoding	Latency and cost per output token on generation workloads
Compilation	Latency and scale-out time, at no accuracy cost
Optimisation jobs	Cost money themselves. You pay for the optimisation instance

ON THE EXAM

The last row matters: an optimisation job is a real charge, so it needs enough traffic afterwards to repay itself.

Combining optimisation techniques

- They compose. AWS's own example combines **INT4-AWQ quantization with speculative decoding**.
- Compilation pairs naturally with either, since it targets the hardware rather than the weights.
- Each addition makes the quality question harder to attribute. Measure after each step.
- Order matters for diagnosis: apply one, evaluate, then apply the next.
- Or take a pre-optimized version, where AWS has already made these choices.

ON THE EXAM

Module 15's discipline again: change one thing, measure, then change the next. Two changes at once and you cannot attribute either.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Container will not start on Graviton	Image built for x86, not Arm64
Model will not deploy to Inferentia	Not compiled for Neuron
Speculative decoding gives no speed-up	Low acceptance rate. Draft model mismatched
Quality dropped after optimisation	Quantization. Compilation does not lose accuracy
Slow to add capacity during a traffic spike	Model load time. Fast model loading, or compilation
GPU instance no faster than CPU	Batch too small, or the workload is not parallelisable
Bigger GPU did not help	The bottleneck was preprocessing or I/O, not the model

ON THE EXAM

Row four is a useful diagnostic: if quality fell, quantization is the suspect, because compilation is documented as accuracy-neutral.

Measure first. Every lever here is cheap to evaluate and expensive to apply blindly.

A developer tools company runs a code-completion model on GPU endpoints. Output is highly predictable, being mostly syntactically constrained code. Inference cost per generated token is the stated problem.

Quality must not drop, because suggestions are shown directly to developers and a wrong completion is worse than a slow one.

Which optimisation should they apply?

A. INT4-AWQ quantization, to reduce the model size and speed up inference

B. Move to a larger GPU instance to increase throughput

C. Fast model loading, to reduce scale-out latency

D. Speculative decoding, with a draft model from the same family

◆ PAUSE AND DECIDE

Which optimisation should they apply?

A INT4-AWQ quantization, to reduce the model size and speed up inference

Quantization trades some accuracy, and the scenario states quality must not drop. It would work and it violates the stated constraint.

B Move to a larger GPU instance to increase throughput

Raises cost per hour to address cost per token. It also does nothing about the sequential bottleneck in generation.

C Fast model loading, to reduce scale-out latency

Addresses load time, not generation cost. Nothing in the scenario mentions scaling or cold starts.

D Speculative decoding, with a draft model from the same family

Predictable output is exactly its case, and the target model verifies every token, so quality is preserved. It directly reduces cost per output token.

THE SIGNAL

Two facts choose it: predictable output raises the acceptance rate, and quality-neutrality is the constraint that eliminates quantization.

A team is deploying a model to a real-time endpoint for the first time. They do not know which instance type suits it, and traffic will be roughly 200 requests per second with a 100 millisecond latency target.

They want a configuration they can defend, with measured evidence rather than a guess copied from a previous project.

What should they do?

A Deploy on the same instance type as their last model and monitor CloudWatch

B Use inference recommendations to get validated, deployment-ready configurations

C Deploy on the largest GPU instance available, then downsize later

D Run a quantization job first, then choose the smallest instance that fits

◆ PAUSE AND DECIDE

What should they do?

A Deploy on the same instance type as their last model and monitor CloudWatch

Exactly the habit the scenario says they want to replace. It produces evidence eventually, after running the wrong configuration in production.

B Use inference recommendations to get validated, deployment-ready configurations

It analyses the model and workload, evaluates instance types, and returns configurations with real performance metrics. AWS calls it the recommended approach for most customers.

C Deploy on the largest GPU instance available, then downsize later

Expensive, and downsizing later rarely happens once something works. It also assumes a GPU is needed at all.

D Run a quantization job first, then choose the smallest instance that fits

Optimises before measuring, and accepts an accuracy cost that may be unnecessary. Sizing comes first.

THE SIGNAL

'We do not know which instance type' is the Inference Recommender signal, and it appears in scenarios almost word for word.

A team wants to serve a large language model on a single GPU instance, but the model does not fit in that instance's accelerator memory. The next instance size up triples the hourly cost.

Some reduction in output quality is acceptable, provided it is measured. Latency should ideally improve as well.

What should they apply?

A Compilation, to optimise the model for the target GPU

B Speculative decoding, using a smaller draft model

C Quantization, then evaluate the quality impact

D Fast model loading, to reduce memory pressure at load time

◆ PAUSE AND DECIDE

What should they apply?

A. Compilation, to optimise the model for the target GPU

Improves performance without accuracy loss, and does not meaningfully reduce the memory footprint. The model still will not fit.

B. Speculative decoding, using a smaller draft model

Adds a second model to memory rather than reducing what is needed. It makes the capacity problem worse, not better.

C. Quantization, then evaluate the quality impact

Fewer bits per weight means a smaller footprint, so the model fits. Latency improves too, and the stated tolerance for measured quality loss permits it.

D. Fast model loading, to reduce memory pressure at load time

Reduces how long loading takes, not how much memory the loaded model occupies. The constraint is capacity, not time.

THE SIGNAL

B is worth noticing: speculative decoding needs BOTH models resident, so it adds memory pressure rather than relieving it.

A team enables speculative decoding on a generation workload. They pair their target model with a small, fast draft model from a different vendor, chosen because it had the lowest latency of anything available.

After deployment, throughput is marginally worse than before and cost has risen. No errors are reported.

What is the most likely cause?

A The draft and target models do not share a tokenizer and vocabulary, so acceptance is poor

B num_speculative_tokens is set too low, so too few tokens are proposed

C Speculative decoding requires compilation to be applied first

D The draft model is too fast, so it outruns the target model's verification

◆ PAUSE AND DECIDE

What is the most likely cause?

A The draft and target models do not share a tokenizer and vocabulary, so acceptance is poor

Speculation operates on token IDs verified by the target. A different vendor almost certainly means a different tokenizer, so rejected tokens cost work and buy nothing.

B num_speculative_tokens is set too low, so too few tokens are proposed

A real tuning knob, and it would reduce the gain rather than reverse it. Throughput got worse, which points at rejection, not conservatism.

C Speculative decoding requires compilation to be applied first

They are independent techniques. Compilation helps, and is not a prerequisite.

D The draft model is too fast, so it outruns the target model's verification

Not a failure mode. The draft proposes, the target verifies, and the sequence is controlled. Speed of the draft model is the point of it.

THE SIGNAL

Worse than before is the clue. Speculation that is rejected costs two models' work for one model's output.

What to take from this module

- **Compilation is accuracy-neutral. Quantization is not.** That distinction decides several questions.
- **Speculative decoding needs a shared tokenizer**, and ideally the same model family.
- **Quantization reduces memory. Speculative decoding increases it**, because both models are resident.
- **Inference recommendations is the AWS-recommended route**, and 'we do not know which instance' is its signal.
- **Inferentia and Trainium need Neuron compilation**, and repay through sustained volume.

ON THE EXAM

Module 23 covers scaling these endpoints: policies, metrics, cooldowns and scale-to-zero.